UNIVERSITY OF BEDFORDSHIRE FOREIGN TRADE UNIVERSITY

---------***-------- ---------***--------

BUSINESS ANALYTICS

Unit Code: CIS051-3

Assignment 1: Data Mining Solutions for Direct Marketing Campaigns

Student Name	:	**PHAM HOANG HAI VAN**
Student ID	:	**2524618**
Email	:	**PhamHoangHai.Van@study.beds.ac.uk**
Unit Coordinator	:	**Dr Vitaly Schetinin**

Vietnam, 2025

TABLE OF CONTENTS

1. Introduction 1

2. Designing a Solution 2

2.1 Exploratory Data Analysis 2

2.2 Data Cleaning and Feature Engineering 7

2.3 Modeling Approach 8

3. Experiments 11

3.1 Baseline Results 11

3.2 Grid Search Optimisation 14

3.3 Cost-Sensitive Threshold Tuning 16

3.4 ROC Curve Comparison 18

3.5 Interpretation and Feature Insight 19

4. Conclusion 20

References 22

Appendix 23

Appendix A: Tables and Analytical Results 23

Appendix A1: Tables and Analytical Results 23

Appendix A2: Descriptive Statistics for All Variables 26

Appendix B: Python Scripts 26

Appendix B1: Exploratory Data Analysis (EDA) 27

Appendix B2: Data Cleaning and Feature Engineering 28

Appendix B3: Model Design, Optimisation, and Cost-Sensitive Threshold Tuning 30

LIST OF FIGURES

Figure 2.1-1:Distribution of target variable ‘y’ 3
Figure 2.1-2: Histograms of numerical variables 5
Figure 2.1-3: Bar charts of categorical variables 7
Figure 3.1-1: Confusion Matrix of Decision Tree and Logistic Regression 13
Figure 3.1-2: ROC Curve Baseline Models (DT vs LR) 14
Figure 3.2-1: Confusion matrix of DT and LR after Optimisation 16
Figure 3.4-1: ROC Curve Comparison: Baseline vs Best Models for Decision Tree and Logistic Regression 18
Figure 3.5-1: Feature importance plot for Decision Tree 19
Figure 3.5-2: Coefficient bars for Logistic Regression (top positive and negative predictors) 20

LIST OF TABLES

Table 2.1-1: Descriptive Statistics (Numerical Variables) 3
Table 2.3-1: Theoretical Confusion Matrix 9
Table 3.1-1: Baseline Model Performance Comparison between Decision Tree and Logistic Regression 12
Table 3.2-1: Model performance comparison after hyperparameter optimisation. 15
Table 3.3-1:Cost Matrix (FP, FN, TP, TN) 17
Table 3.3-2: Threshold optimisation top result for Decision Tree and Logistic Regression. 17
Table 3.5-1: Model Comparison after Cost-Sensitive Optimisation 20

LIST OF FOMULARS

Formula 2.3-1: 10
Formula 2.3-2: 10
Formula 2.3-3: 10
Formula 2.3-4: 10
Formula 2.3-5: 10
Formula 2.3-6: 10
Formula 2.3-7: 11

1. Introduction

One of the influential channels to advertise the banking products and influence the customer decisions is telemarketing. Face-to-face and direct communication through the phone assists banks to create awareness, motivate their recipients, and form a continuous relationship. The findings presented by Moro, Cortez, and Rita (2014) revealed that information gathered through telemarketing campaigns can provide vital information on the pattern of customer response. The adoption of predictive models enhanced the accuracy in determining clients who would therefore accept the term-deposit offers.

Moro et al. (2014) established the foundations of prediction of telemarketing by training the logistic regression and decision tree models on the Portuguese Bank data, with an AUC of approximately 0.79. Their study established that data-based modelling improves precision in targeting and efficiency of a campaign. This was later elaborated by Ilham et al. (2019) who tested Random Forest and SVM with AUC scores of 0.84 to 0.88. Their results demonstrated that the ensemble and kernel-based algorithms are more predictive than linear models. Feng et al. (2022) made a contribution to this direction by developing META-DES-AAP, a meta-learning model that enhanced accuracy and profit margins. This model fitted the complicated customer-related data and minimized cost inefficiency. More recently, Zaki et al. (2024) suggested a hybrid model to integrate Random Forest, KNN, and SGD Classifier, which presented the accuracy of 87.5. The hybrid methodology balanced accuracy, recall and cost of marketing, indicating that a combination of ensemble and gradient-based models facilitates accuracy and efficiency of resources.

The development is evident in these studies. The initial interpretable models developed into adaptive hybrid systems, which are concerned with accuracy, interpretability, and cost to an equal degree. But there is still a gap: to provide a clear and practical models that would explain and maximize marketing outcomes within realistic cost requirements.

Thus, this report creates a data-mining application that will forecast the possible acceptance of a term deposit proposal offered to a client and aiming at minimizing the marketing expenses. The dataset used in the analysis is the Portuguese Bank Marketing (UCI, 2012) and the applied models are Decision Tree and the Logistic Regression models. The reason that both algorithms are selected is their simplicity, speed and the ability to deal with imbalanced data that is prevalent in telemarketing.

It is expected that the outcomes of this report will enhance marketing decision-making by

predicting the response more accurately; making it more focused and lessening the spamming. The report is organized into four sections, which include the Introduction, Designing a Solution, Experiments, and Conclusion.

2. Designing a Solution

2.1 Exploratory Data Analysis

The data employed in the present study, Bank Additional, consists of 41,188 records and 20 variables, which refer to client demographics, the features of the campaign, and macroeconomic parameters. They are personal factors (age, employment, matrimonial status and education) and financial factors (mortgage, history of prior contacts, and annual account balance). The external economic conditions are measured using the macroeconomic indicators like emp.var.rate, cons.conf.idx, and euribor3m, which influence the level of confidence by customers in subscribing to term deposits. All the variables are described in Appendix A1.

The second step was the Exploratory Data Analysis (EDA) conducted prior to the model development in order to determine the structure and quality of the dataset. This is necessary to detect the distribution patterns, outliers, and relationships that could influence the predictive accuracy. It makes sure that subsequent modelling phases are made on valid and interpretable data. EDA offered a comprehensive insight into the consistency of internal data and showed the interactions of customer characteristics with the variables of the economy. This knowledge became the analytical centrism of all the further modelling operations.

Starting with variable (y), the target variable (y) will be the yes or no factor of whether the client has subscribed to a term deposit. This distribution is very skewed as illustrated in Figure 2.1-1: approximately 11% of the clients were subscribers and almost 89% were non-subscribers. The imbalance suggests that the non-subscribers will take control of the dataset, which will result in biased predictions favoring the majority category. This imbalance is a common problem in marketing analytics, in which bank are trying to understand a small but valuable group of likely buyers. With uncorrected imbalanced dataset, high accuracy and low recall of positive cases can be achieved which implies that potential customers are missed. To control this, subsequent analysis steps used cost sensitive modelling and adjustments to the threshold so as to enhance a recall of the minority category without losing control of the costs of the campaign.

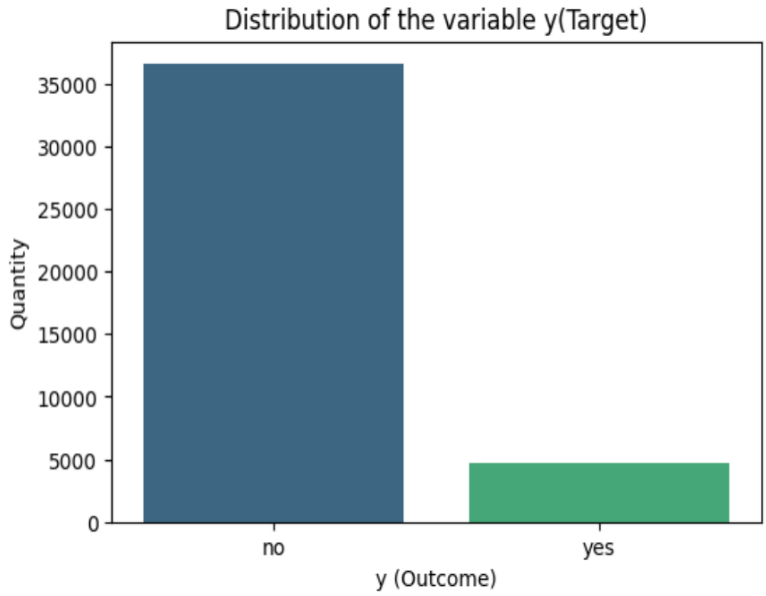


Figure 2.1-1:Distribution of target variable ‘y’

In addition to the analysis of the target distribution, the predictor variables describing customer attributes and campaign performance were also given more attention. These predictors were categorized into two including numerical and categorical. In case of numerical variables, mean, standard deviation and range were calculated as the descriptive statistics to summarise the customer demographics, campaign intensity and pertinent macro economic conditions. This quantitative summary assisted the interpretation of behavioural patterns that determine deposit subscriptions. The entire descriptive statistics can be seen in Appendix A2 whereas Table 2.1-1 shows an overview of the key numerical variables used in the study.

Table 2.1-1: Descriptive Statistics (Numerical Variables)

Variable	Count	Mean	Min	Max	Std
age	41.188	40.024	17.000	98.000	10.421
duration	41.188	258.285	0.000	4918.000	259.279
campaign	41.188	2.568	1.000	56.000	2.770
pdays	41.188	962.475	0.000	999.000	186.911
previous	41.188	0.173	0.000	7.000	0.495
emp.var.rate	41.188	0.082	-3.400	1.400	1.571

cons.price.idx	41.188	93.576	92.201	94.767	0.579
cons.conf.idx	41.188	-40.503	-50.800	-26.900	4.628
euribor3m	41.188	3.621	0.634	5.045	1.734
nr.employed	41.188	5167.036	4963.600	5228.100	72.252

Overall, the descriptive findings are characterized by a number of interesting trends. The mean age is 40 years, which implies the majority of clients are middle-aged, whereas the upper age limit is 98, which means that the data set has a wide demographic group. The variable campaign, which is used to measure the number of contact attempts in the current campaign, has a mean value of 2.57 with a maximum of 56, which means that most of the clients were only contacted a few times, but there was a small group that was provided with regular follow-up calls.

The most notable variable is pdays with most of the observations having 999, indicating that these clients have never been contacted before and it is practically an artificial figure and not a true one. On the same note, previous (mean = 0.173) indicates that most clients had some (a little or none at all) history of interaction with past campaigns. At the macroeconomic level, the indicators like the emp.var.rate, cons.price.idx, and euribor3m vary in realistic levels, which indicate the changing employment rates, consumer confidence and interest rates in the market at the time of the campaign.

In addition to these numerical summaries, visualisation was employed so as to have a more in-depth insight into the distributions of variables. Histograms were drawn to identify skewness, outliers and possible abnormalities that could influence the performance of the model. Some of the numerical variables, like age, campaign and previous, as shown in Figure 2.1-2, are skewed to the right indicating that the majority of the customers are in the middle age range and were relatively few receiving contact calls. In the meantime, such economic indicators as euribor3m and emp.var.rate exhibit visible variations that mirror the macroeconomic factors that might affect the decision of customers and responsiveness to marketing.

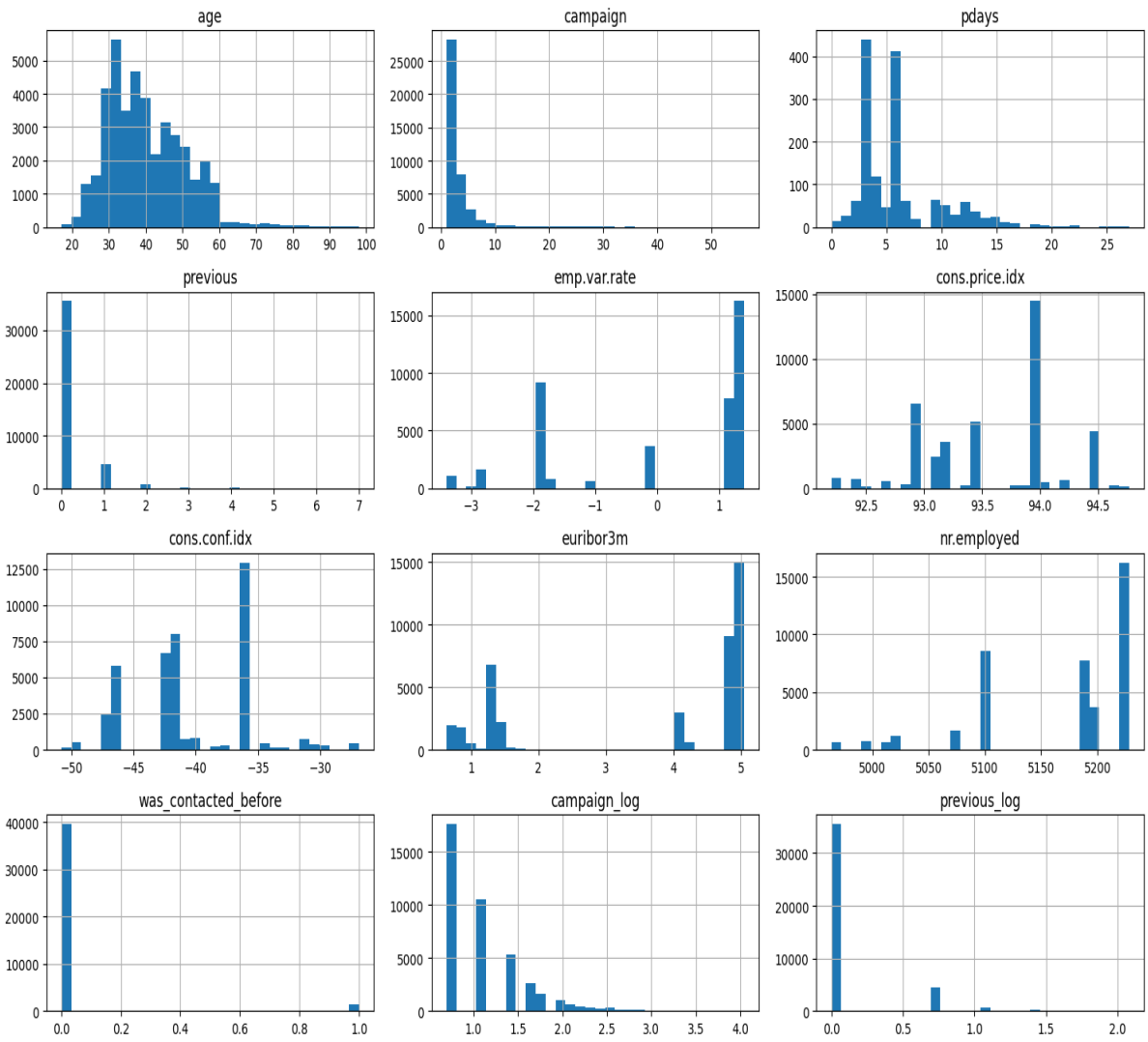
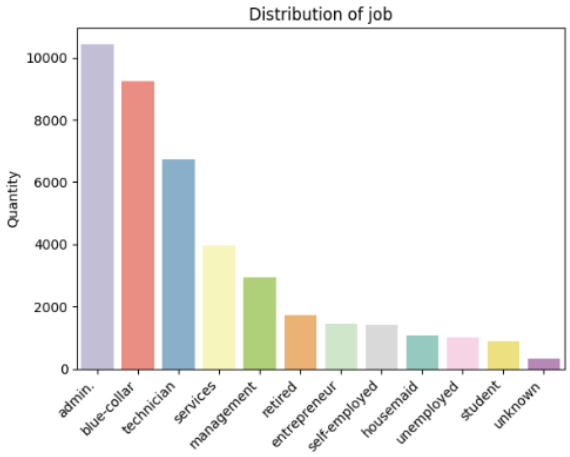


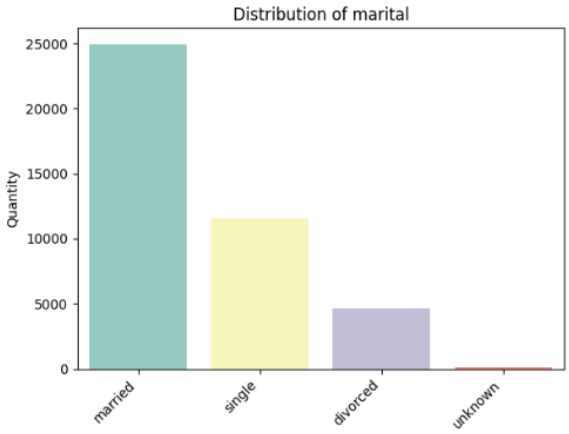
Figure 2.1-2: Histograms of numerical variables

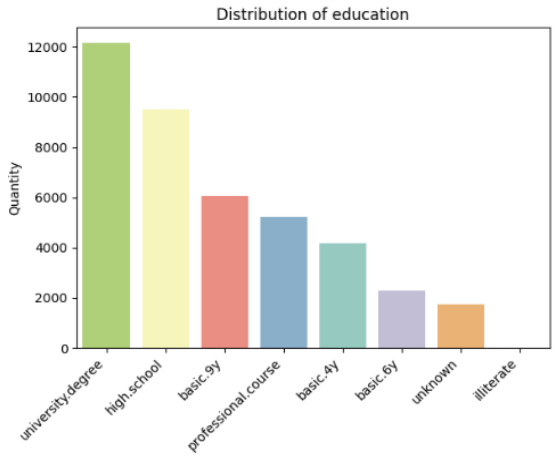
Then, to gain a more objective interpretation of the proportion of each category and identify possible data quality issues, Categorical variables were subsequently examined with the help of frequency tables and bar chart visualisations. In this analysis, it was possible to make a number of observations. As an example, there were almost no variations in certain variables, including default, which was dominated by one value (no), thus restricting their modelling value. Also there were several variables with unknown values which when preprocessing they should be treated as missing values to eliminate a false interpretation. In some instances, including job, education, and pout outcome, there were some few categories which had less than 1 percent of the total observations. These are the rare categories that were subsequently classified as a larger label of Other to facilitate the model format and enhance stability.

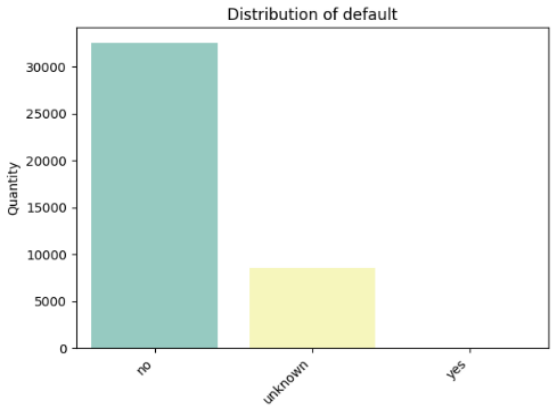
The distribution of the categorical variables is provided in the Figure 2.1-3, which shows that the dataset mainly comprises of married people with secondary or University education, with

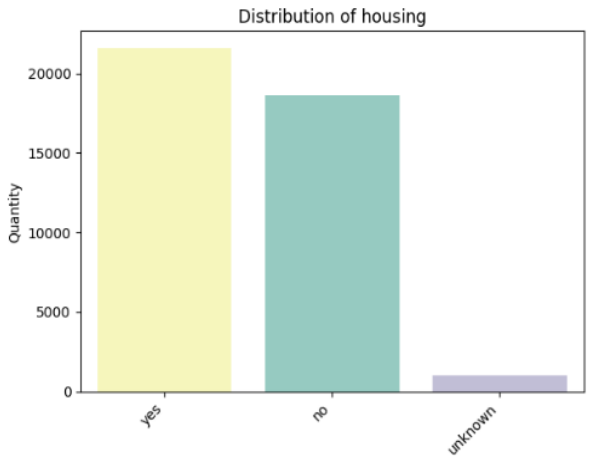
stable employment and having a housing loan. The majority were done through mobile phones, and the success of the past campaigns was mostly non-successful, and this was in line with the low percentage of term-deposit subscriptions recorded in the target variable.

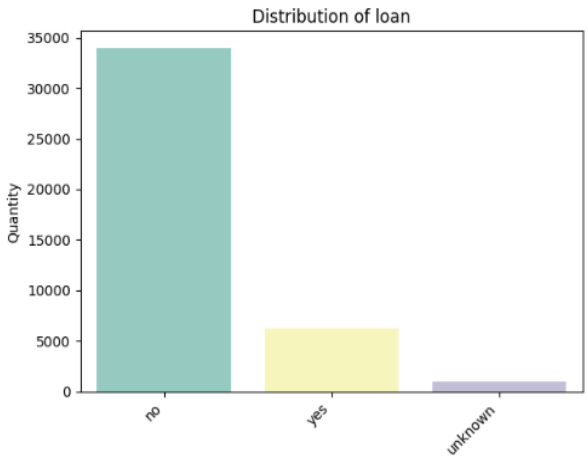












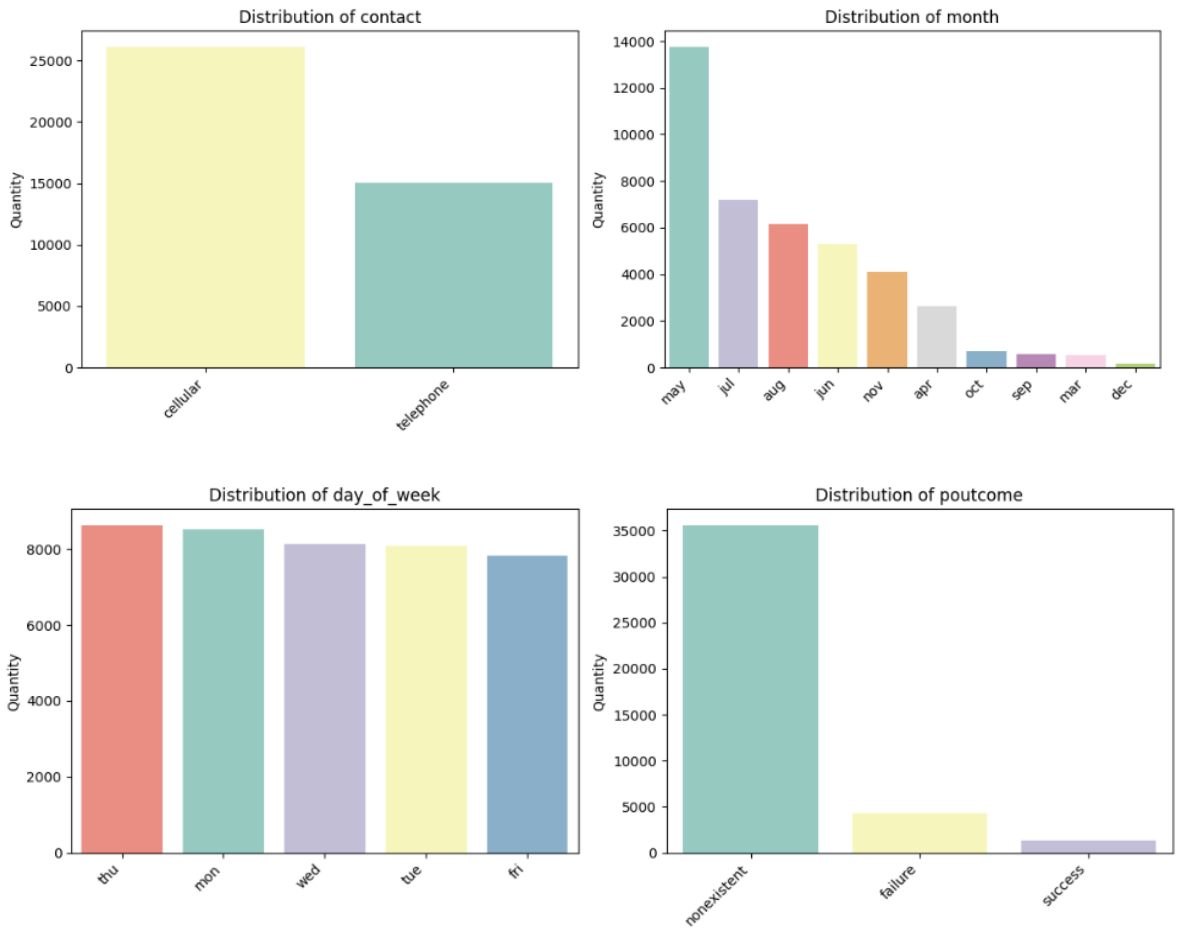


Figure 2.1-3: Bar charts of categorical variables

To conclude, the exploratory analysis helped to obtain a detailed picture of the nature of the dataset and its limitations. The target variable was highly unbalanced, whereas a number of input variables had missing or low-variance values that would need some special consideration. In addition, the skewed nature of numerical variables and outliers were present, and the categorical variables had to be grouped to minimize the sparsity. The next section has used these insights to inform the process of data cleaning and feature engineering.

The Python scripts that were to generate the descriptive statistics, histograms, and categorical plots are all provided in the Appendix B1.

2.2 Data Cleaning and Feature Engineering

Following the exploratory phase, a number of data cleaning operations were performed to enhance the quality of data and guarantee the quality performance of the model. To begin with, any cases identified as unknown were substituted with NaN (missing values) so that any uncertain cases would be treated as needed instead of being perceived as meaningful data. Categorical variables were imputed with the ‘most frequent category’ where missing values

occurred whereas numerical variables were imputed with the ‘median’ because it is less susceptible to extreme values. Duration and default were then eliminated in the dataset. The duration variable that is expressed as time spent on a call was omitted to prevent data leakage since such information would be obtained after the call has taken place. The default variable was discarded because, of the over 95 percent of its values, its values were no, which essentially offers no variance and predictive power. Also, categories with few counts of job, education and poutcome were combined together as a single category of other to reduce the sparsity and enhance the strength of the model.

Following the cleaning, feature engineering was done to trim and build on the remaining variables to enhance model learning. The pdays attribute (number of days since the last contact) was converted into a binary measure called wascontactedbefore (1 = previously contacted, 0 = not contacted). This change made the interpretation process easier and removed the confusion that would occur because of the pseudo-values like 999. To solve skew and huge numerical variance, logarithmic transformed variables were used where the histogram analysis indicated high skew values such as campaign and previous. Such changes minimized the impact of extreme values and made the numbers on features more equal. Z-score scaling was then applied to all the numerical variables to standardise them so that they have similar ranges, and no individual variable will control the training process. One-hot encoding was used to encode categorical variables to enable the algorithms to be capable of interpreting non-numeric data.

After data preparation was done the refined dataset was split into training (80 percent) and testing (20 percent) subsets through stratified sampling to retain the imbalance of classes of the target variable (y). This strategy was followed to make sure that the proportion of subscribed and non-subscribed clients in both subsets was equal, giving an equal opportunity to the models to be evaluated equally and consistently. In this set of ordered cleaning and transformation processes, the data was made into a comprehensive, bias-free and fully prepared to be trained format, which serves as a strong backbone to further predictive modelling. The corresponding Python code to implement these processes is added at the Appendix B2 so that the processes are reproducible.

2.3 Modeling Approach

Continuing on the research conducted earlier in Introduction, the study will use two complementary supervised learning algorithms, Decision Tree and Logistic Regression, to answer the question whether a client is going to subscribe to a term deposit. The algorithms

have been chosen based on their simplicity, computing power and capability to perform binary classification with the presence of class imbalance. The Decision Tree finds non-linear, complicated interactions among features using hierarchical rules whereas the Logistic Regression presents a well-calibrated, consistent linear baseline. The combination of them forms a balanced framework that integrates predictability with interpretability.

The Decision Tree algorithm divides the data into smaller groups by choosing the feature that gives the greatest information gain during each split. The result of this process is a sequence of rule-based decisions that can be stated in the plain language, e.g. the more a client was contacted via cellphone and had been reached previously, the higher the chances of them subscribing. Its transparency and visual output are suitably applicable in any marketing related task where managers are required to convert model results into actionable campaigns.

Logistic Regression is used to estimate the probability of subscribing by providing a logistic (sigmoid) transformation of weighted linear sum of the input variables. The output will be between 0 and 1, which will be a measure of the likelihood that each client will be accepted. This design, a probability-based one, is a complement to the Decision Tree; it is a design that generates smoother decision boundaries and reduces overfitting. The regression coefficients also give first-hand information about the effect of each variable on the outcome, which justifies clarifiable, data-driven marketing decisions.

The two models were optimised with regards to parameters to provide a balanced trade-off between the interpretability, sensitivity to positive cases, and generalisation. They were then measured in terms of their prediction accuracy and business value in terms of their performance.

To Evaluation Framework: Model performance has been measured in terms of several metrics: Accuracy, Precision, Recall, F1-score, ROC-AUC, PR-AUC and Balanced Accuracy. All metrics provide a variant of measure of model effectiveness. These numbers are based on the confusion matrix(Table2.3-1) which summarises the four potential scenarios, the True Positives (detected a subscriber correctly), the True Negatives (detected a non-subscriber correctly), the False Positives (detected as a subscriber and not a non-subscriber correctly) and the False Negatives (detected as a subscriber and missed by the model).

Table 2.3-1: Theoretical Confusion Matrix

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Mathematically, the metrics are defined as follows:

Formula 2.3-1:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

This is the percentage of correct predictions. High Accuracy implies that the majority of the classifications are accurate, however, in unbalanced datasets, this measure, on its own, can be misleading because it is dominated by the majority class.

Formula 2.3-2:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Precision will determine the number of customers that were actually forecasted to subscribe. A larger Precision in marketing terms would imply that fewer calls are wasted to clients who are not interested.

Recall captures the proportion of actual subscribers correctly identified. This metric is prioritised in telemarketing contexts because missing potential clients (false negatives) translates to lost opportunities.

Formula 2.3-3:

$$\text{Recall (TPR, Sensitivity)} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

The F1-score provides a harmonic balance between Precision and Recall, especially useful when both metrics must be considered simultaneously to evaluate campaign efficiency.

Formula 2.3-4:

$$\text{F1} - \text{score} = 2 \times \frac{(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})}$$

Specificity is used to determine how the model correctly identifies non-subscribers, whereas Balanced Accuracy balances out class imbalance by averaging the results between positive and negative classes.

Formula 2.3-5:

$$\text{Specificity (TNR)} = \frac{\text{TN}}{\text{TN} + \text{FP}}$$

Formula 2.3-6:

$$\text{Balanced Accuracy} = \frac{\text{Recall} + \text{Specificity}}{2}$$

ROC-AUC measures the total separability of the model with respect to various thresholds with a large area indicating that the model distinguishes better between subscribers and non-subscribers. Equally, PR-AUC (Precision-Recall AUC) is an evaluation of the space below the Precision, Recall curve, which is more informative in highly imbalanced datasets.

Formula 2.3-7:

$$\mathrm{ROC-AUC} = \int \int_0^1 \mathrm{TPR(FPR)dFPR}$$

Recall and ROC-AUC are considered to be prioritised in the analysis of telemarketing since it is likely to be more expensive to miss potential clients (FN) than to contact uninterested ones (FP). Meanwhile, Precision, F1-score, and cost-sensitive measures guarantee the effective utilization of campaign resources. The entire setup of the configuration and setup of parameters is described in the Experiments section below.

3. Experiments

The entire experimentation in this paper was done on the Python programming language on Google Colab and the advantage of integrated libraries (scikit-learn, numpy and matplotlib) was used to model, evaluate and visualise.

3.1 Baseline Results

Based on the model specification in Section 2.3, Decision Tree (DT) and Logistic Regression (LR) models were trained on the processed dataset to determine the baseline performance of each.

To ensure interpretability and prevent overfitting, the Decision Tree baseline was set with a criterion of entropy, maxdepth of 4, and minsamplesleaf of 7. A LBFGS solver was used to train the Logistic Regression with maxiter=2000, L2 regularisation to maintain a stable convergence and a balance of weights of the imbalanced data as the classes.

The purpose of this stage was to compare the ability of each model to classify customer response prior to any hyperparameter tuning or threshold adjustment to create a baseline with which to optimise.

Table 3.1-1: Baseline Model Performance Comparison between Decision Tree and Logistic Regression

Model	**Accuracy**	**Precision**	**Recall (TPR)**	**F1-score**	**ROC-AUC**	**PR-AUC**
Logistic Regression (baseline)	0.901	0.698	0.219	0.333	0.801	0.464
Decision Tree (baseline, entropy)	0.899	0.690	0.202	0.312	0.786	0.417

As the base findings in Table 3.1-1 indicate, both of the models obtained similar levels of accuracy (≈0.90), which proves their constancy in terms of the quality of prediction on the dataset. Nevertheless, different trade-offs can be observed when specific measures are analyzed. There was a little better Recall (0.219) and ROC-AUC (0.801) in Logistic Regression, which means better discrimination of potential subscribers and non-subscribers. On the other hand, the Decision Tree was slightly more accurate (0.690) but less Recall (and therefore more conservative) suggesting that it was less risky and was more accurate in classifying fewer customers as may subscribe. Generally, the Logistic Regression was more calibrated and capable of generalisation and the Decision Tree was easier to interpret, which could be used to extract decision-rules.

These observations would further be supported by a closer examination of the confusion matrices (Figure 3.1-1). Both models made the needed variations of detection of non-subscribers, as they both had the necessary percentage of True Negatives. Nevertheless, they both had difficulties with determining all possible subscribers, as the number of False Negative (FN) was high (741 in DT and 725 in LR). Such misclassifications are a wasted marketing opportunity, in which the potential customers were not defined. Nevertheless, both models performed a trade-off betweenTrue Positive and True Negative predictions and Balanced Accuracy rates are 0.595 with DT and 0.603 with LR, which confirms that the two models perform well as baselines.

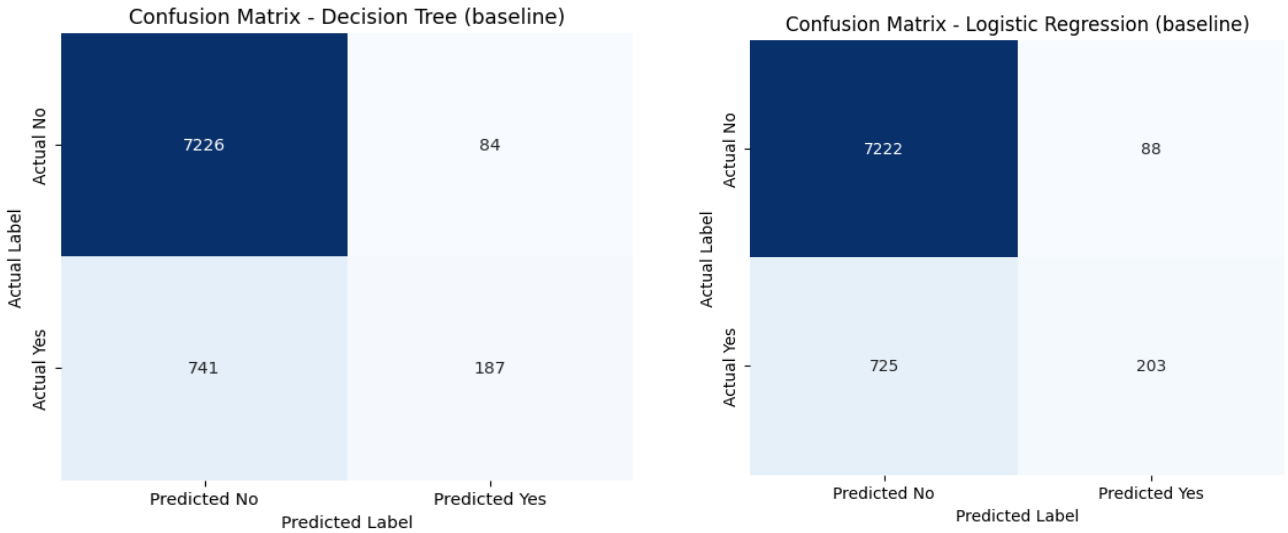


Figure 3.1-1: Confusion Matrix of Decision Tree and Logistic Regression

In order to further discriminate the two models, the ROC curves in Figure 3.1-2 show how the two models separate the positive and negative classes at different probability thresholds. The two curves are well above the diagonal reference line, which validates the performance that is significantly beyond the level of random chance (AUC between 0.79 and 0.80).

Logistic Regression had a slightly better AUC of 0.80, which implies that it has more reliable probability calibration and class separation. This finding indicates that the model ranks the clients in a better manner based on their probability of subscribing. The Decision Tree scored AUC of 0.79, a little less and yet, a strong score that demonstrates the reliable discriminative performance of the rule-based, but simpler structure of this model.

Although the Logistic Regression proves to be slightly better in predictive consistency, the Decision Tree is a better tool to be used in the management as it is interpretable and transparent. Learning the process of making predictions is important in translating model results to marketing activity. The results of these ROC provide a good baseline on the basis of which subsequent optimisation by adjusting the parameter and cost-sensitive threshold adjustment is supplied.

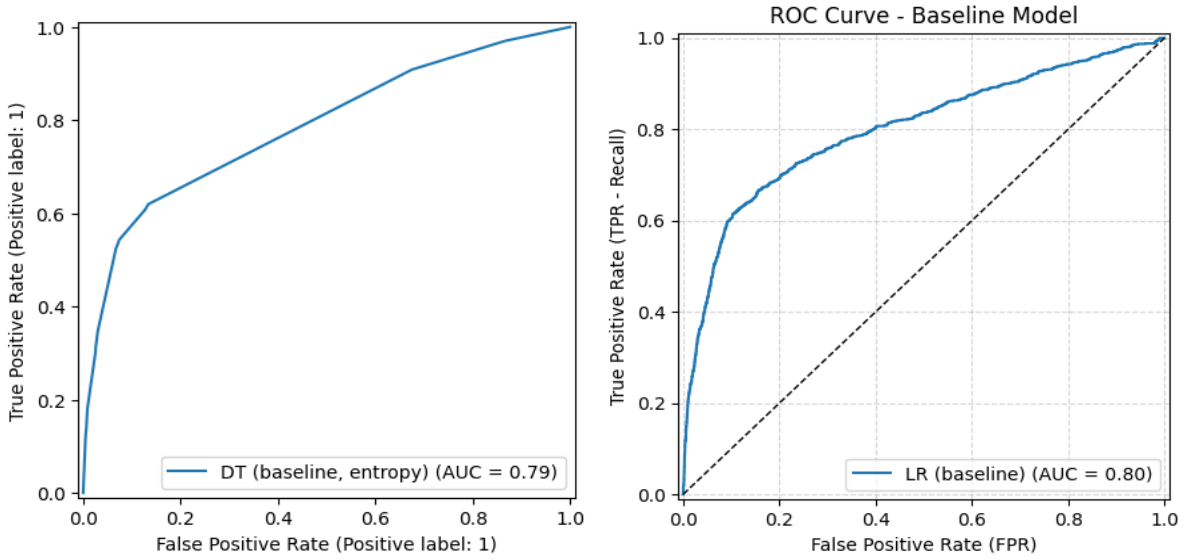


Figure 3.1-2: ROC Curve Baseline Models (DT vs LR)

In conclusion, both the models make the solid background base. Logistic Regression was a little more recall-accurate and AUC, but the decision-tree was a lot easier to understand. These findings help to further optimise the hyperparameter tuning and the cost-sensitive adjustments of the threshold to enhance the recall and reduce the marketing cost inefficiencies in subsequent stages.

The Python code used for this baseline analysis is available in Appendix B3.

3.2 Grid Search Optimisation

To enhance predictive accuracy and balanced generalisation, the hyperparameter optimisation was carried out with the assistance of the method of the Grid Search that is complemented with the cross-validation. It was a systematic experiment with parameter combinations with predefined parameters to identify a parameter combination that would achieve the best ROC-AUC score on validation data. This was to maintain that every model is refined and the complexity, interpretability and recall are balanced and balanced in identifying potential subscribers in an unbalanced data.

The parameter grid in the case of Decision Tree (DT) model was concerned with complexity and overfitting of the tree. It applied various maxdepth (4 to 16) to limit the growth of branches as well as various ccpalpha (to 0.003) to apply pruning. Thresholds of minsamplesleaf (2-30), and minsamplessplit (5-80) were also tested to obtain statistical significance and generalisation. Notably, the imbalance between the classes was overcome by adjusting the classweight to

balanced where the minor class was allocated with a similar weight.

The grid search was done in the case of the Logistic Regression (LR) model with the different optimisation solver algorithms (liblinear, lbfgs, saga) and their compatible regularisation penalties (l1, l2). This was accompanied with different strengths of regularisation (C values between 0.05 and 5.0) to ensure overfitting is checked. The influence of the classweight ([None, 'balanced']) was also tested with the same search to equalize unequal numbers of classes between them, and max_iter was fixed to 2000 (to allow the search to run long enough to converge).

Table 3.2-1: Model performance comparison after hyperparameter optimisation.

Model (Optimised)	**Accuracy**	**Precision**	**Recall (TPR)**	**F1-score**	**ROC-AUC**	**PR-AUC**
Decision Tree (best, entropy)	0.822	0.351	0.688	0.465	0.808	0.448
Logistic Regression (best)	0.833	0.365	0.646	0.466	0.801	0.459

Each model was trained and validated across these parameter grids, and the configuration with the best ROC-AUC was selected as the final version for further evaluation.

The confusion matrices in Figure 3.2-1 visualise post-optimisation predictions, revealing that both Decision Tree and Logistic Regression reduced false negatives while maintaining a stable number of true negatives compared to their baseline counterparts.

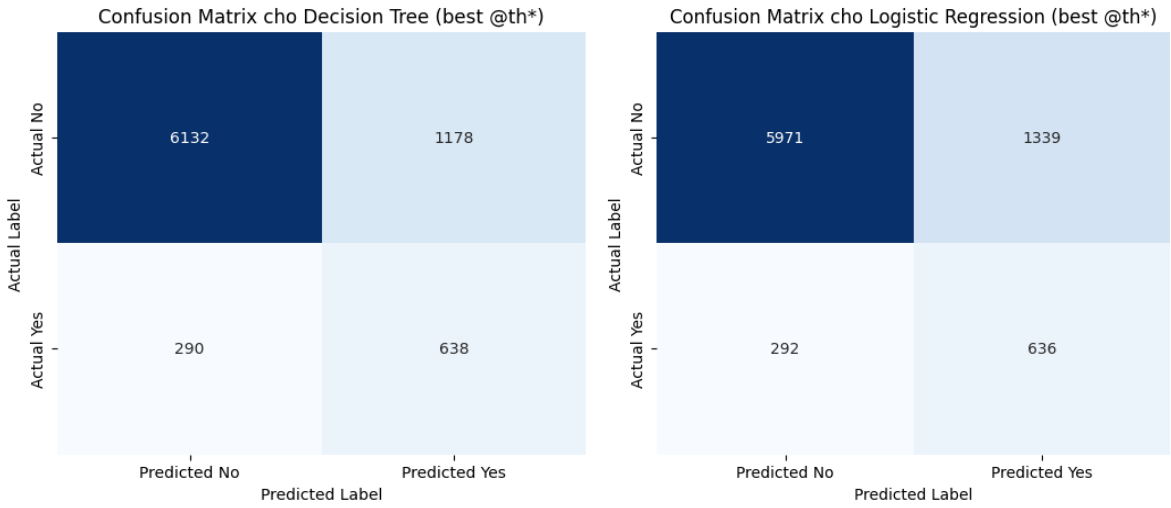


Figure 3.2-1: Confusion matrix of DT and LR after Optimisation

Both models were shown to have a significant increase in recall following tuning as shown in Table 3.2-1 showing increase in sensitivity to potential subscribers. The Decision Tree had a little better recall (0.688 vs. 0.646) whereas Logistic Regression had better overall accuracy (0.833) and calibration (ROC-AUC ≈ 0.80). Despite the similarity in F1-scores of the two models, the high recall of the Decision Tree would yield higher business value since the loss of business due to missed opportunities would be minimized which is of vital importance in marketing where the potential clients are too expensive to have missed.

Finally, the grid search optimisation was also efficient in improving the sensitivity of the model and cost efficiency, which formed the basis of the cost-sensitive threshold tuning analysis presented in Section 3.3.

All the Python scripts that will execute the parameter grids and training processes are reported in Appendix B3 to be reproducible and transparent.

3.3 Cost-Sensitive Threshold Tuning

Building on the optimised model performance from the previous section, this stage introduced a cost-sensitive threshold adjustment to ensure that predictions align not only with accuracy objectives but also with real business outcomes. In marketing campaigns, not all prediction errors carry the same financial impact, missing a potential customer (false negative) is substantially more costly than calling an uninterested one (false positive). Therefore, a cost-sensitive evaluation was performed to determine the optimal probability threshold that minimises overall campaign cost.

To simulate realistic marketing scenarios, a cost matrix was defined to quantify the financial impact of each prediction outcome (Table 3.3-1). The largest penalty, COST_FN = 15.0, represents the high loss associated with missing a potential subscriber, whereas COST_FP = 2.0 corresponds to the smaller cost of contacting an uninterested customer. A successful contact (COST_TP = -4.0) generates a profit, while correctly ignoring a non-subscriber (COST_TN = 0.0) incurs no cost.

Table 3.3-1:Cost Matrix (FP, FN, TP, TN)

Type	**Description**	**Cost**
FP	Called non-interested customer	+2.0
FN	Missed potential customer	+15.0
TP	Successful contact (conversion)	-4.0
TN	Correctly ignored non-subscriber	0.0

This matrix assigns higher penalties to missed opportunities (FN) than to unnecessary calls (FP), reflecting real marketing priorities. By modelling costs in this way, the optimisation process focuses on minimising average campaign expenditure while maintaining practical decision relevance.

Table 3.3-2: Threshold optimisation top result for Decision Tree and Logistic Regression.

Top 1 Thresholds (Min Avg Cost)	**Threshold**	**Avg. Cost**
Decision Tree (best)	0.41	**0.504**
Logistic Regression (best)	0.45	**0.548**

The findings indicate that both models used the lowest average cost at a threshold near to that of conventional 0.5 cut-off, which is around 0.41 in the case of Decision Tree and 0.45 in the case of Logistic Regression. Nonetheless, the Decision Tree was more effective than the Logistic Regression as the average cost of a campaign was lower 0.504, as opposed to 0.548 in Logistic Regression.

3.4 ROC Curve Comparison

This section compares the optimised Receiver Operating Characteristic (ROC) curves to determine the progress in the classification performance after optimisation based on hyperparameter tuning and cost-sensitive threshold adjustment.

ROC curve is a graphic representation of the trade-off between the True Positive Rate (Recall) and the False Positive rate (FPR) at different levels. A curve that is located further upward towards the left side signifies a better capability to differentiate between subscribers and those not.

As it can be seen in Figure 3.4-1, both optimised models were clearly superior to their baseline counterparts. The ROC-AUC of the Decision Tree model was 0.79 and was improved to 0.81 which means that after refining the parameters, the model is more sensitive in identifying potential subscribers. Meanwhile, the Logistic Regression retained its steady AUC at 0.80 with its optimised curve displaying a steeper increase at lower false-positive rates, indicating in essence better efficiency at discovering likely customers early with fewer unnecessary calls.

These findings are affirmative that model optimisation did not only increase the predictive precision but also the general reliability and discriminatory strength. When applied practically in the marketing field, it means that it will lead to more efficient resource allocation, fewer wrongfully classified contacts, and higher conversion rates of prospects identified correctly.

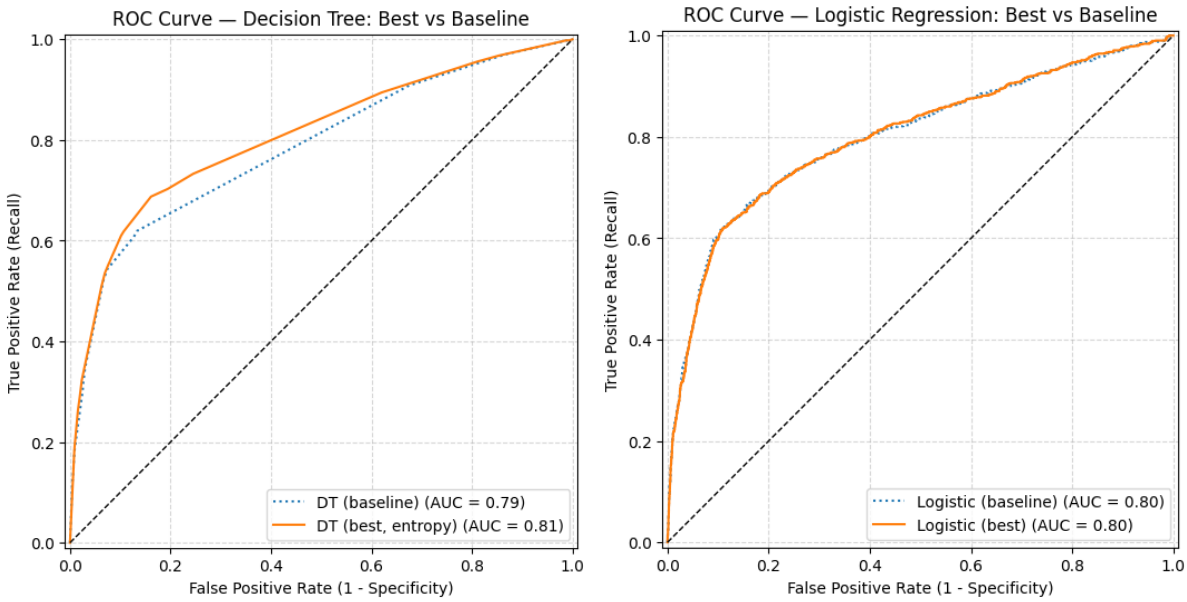


Figure 3.4-1: ROC Curve Comparison: Baseline vs Best Models for Decision Tree and Logistic Regression

All Python code used to generate the ROC curves for both baseline and optimised models is provided in Appendix B3 for reference.

3.5 Interpretation and Feature Insight

To further explain the optimised models and interpret the customer subscription behaviour drivers, this section evaluates the feature importance of each model, and compares the post-optimisation results to find the most adequate solution to be deployed. Figure 3.5-1 and 3.5-2 indicate variables that had the strongest effects on model predictions using visualisations.

In the case of the Decision Tree, the ranking of feature importance (Figure 3.5-1) indicates that the most decisive variable was nr.employed (number of employees), and then comes cons.conf.idx (consumer confidence), euribor3m (interest rate), and wascontactedbefore. The above findings indicate that economic sentiment, stability in the labour market and history of previous contacts are important determinants of a customer subscribing to a term deposit.

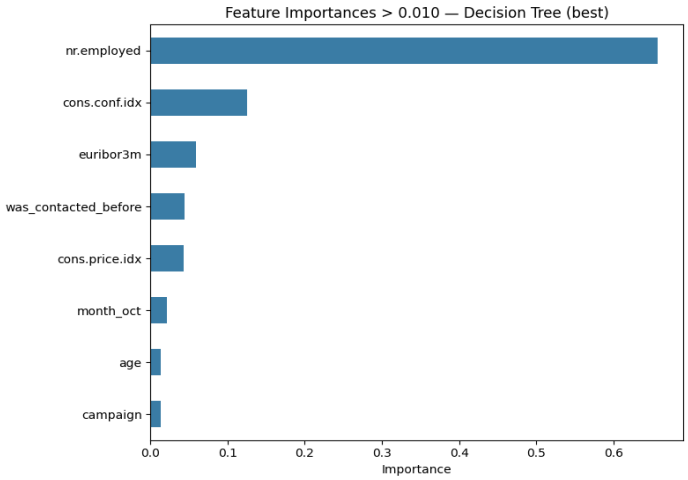


Figure 3.5-1: Feature importance plot for Decision Tree

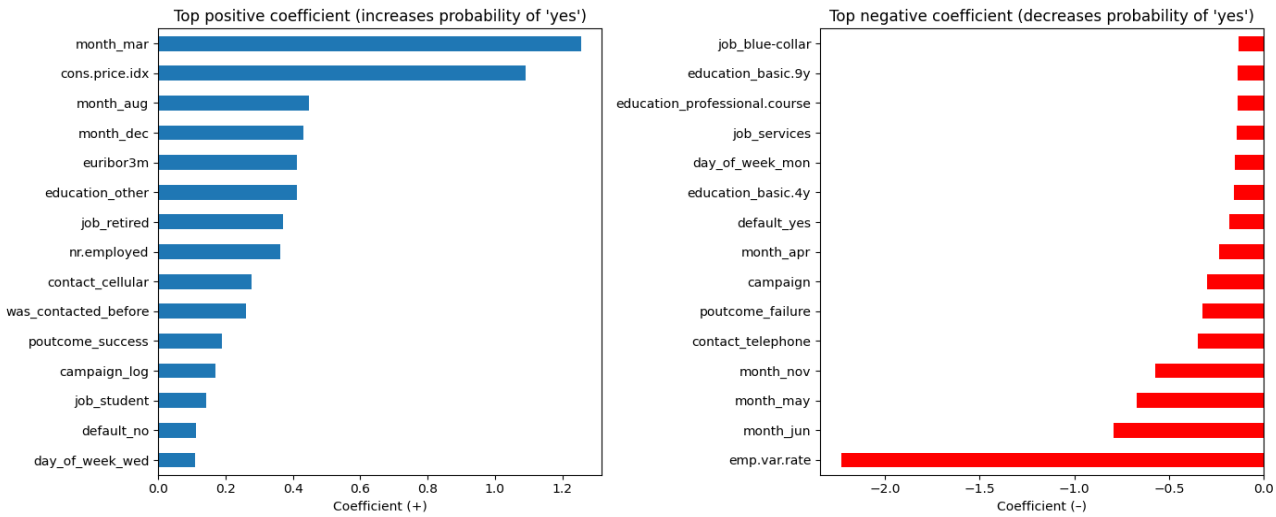


Figure 3.5-2: Coefficient bars for Logistic Regression (top positive and negative predictors)

On the contrary, Logistic Regression can be directly interpreted with the help of coefficient weights (Figure 3.5-2). There are positive predictors like monthmar, cons.price.idx, and euribor3m, which suggest that the favourable market conditions and particular months of campaigns are more likely to engage clients. The negative predictors, such as emp.var.rate, monthjun, and contact telephone, demonstrate that the increase in employment volatility, campaigns in the middle of the year, and using phones as the method of contact are more likely to decrease the success rate of subscriptions.

Table 3.5-1: Model Comparison after Cost-Sensitive Optimisation

Model	**Threshold**	**Accuracy**	**Precision**	**Recall**	**F1**	**ROC_AUC**
Decision Tree (best @th*)	0.410	0.822	0.351	0.688	0.465	0.808
Logistic Regression (best @th*)	0.450	0.802	0.322	0.685	0.438	0.801

Referring to the last comparison of the post-optimisation results (Table 3.5-1), the Decision Tree was better in terms of recall (0.686) and average cost (0.504), which proves that it could better capture potential customers. In spite of the fact that Logistic Regression smootherly calibrated and had the same ROC-AUC (0.801), its cost performance was significantly worse.

Thus, the final model is the Decision Tree model (0.41 threshold). It has a better trade-off among predictive accuracy, business cost-efficient, and interpretability, and hence is the most viable option to use in real-world marketing.

The code and all the related code needed to calculate the feature importance, the weight of coefficients, and the end-model analysis measures are provided in Appendix B3 to facilitate reproducibility and transparency.

4. Conclusion

This research was expected to enhance the effectiveness of bank telemarketing campaign through predictive analytics of customer subscription behaviour at reduced marketing cost. The project applied and tested two interpretable supervised learning models (Decision Tree and Logistic Regression) using Portuguese Bank Marketing data in a cost-sensitive model, which

aims to optimize the tradeoffs between prediction accuracy and resource consumption.

The Decision Tree was the superior of the two algorithms. It got a recall of 0.69 and average campaign cost of 0.504. These results indicate that the model is effective to select a larger portion of the potential subscribers at minimum marketing costs. The decison tree is also very rule-based thus making it fully interpretable hence becoming an effective instrument that can be used by marketing teams that need to have a data-driven and transparent process of decision-making.

The predictive power of the model is comparable to the results of earlier research and is in accordance with those of Moro et al. (2014), who reported Decision Tree as having an AUC of approximately 0.79. This project was focused on simplicity, interpretability, and business implementation, whereas later studies by Ilham et al. (2019) and Zaki et al. (2024) provided higher AUC values, ranging 0.85 to 0.88 using ensemble and hybrid models. The cost-optimisation paradigm taken in this case is a viable complement to the more intricate systems of thought, demonstrating that transparent models do not needlessly yield meaningful cost-efficient predictions in realistic marketing conditions.

Managers-wise, the findings indicate that the banks need to use the predictive insights directly when planning their campaigns by tracking and prioritising employment stability, economic, and past contact history. Targeting these can lower the number of customer outreachs that are unnecessary and the marketing efforts can be channeled to the clients who have the most chances of conversion.

Although these have positive results, there are a number of limitations. The analysis was based on the fixed, past data and tested upon two baseline algorithms. Future directions of this approach would include ensemble and deep-learning methods, incorporation of real-time feedback systems and better cost modelling with live campaign data. Such improvements would see it make adaptive predictive systems that would constantly improve marketing decisions as customer behaviour and market conditions change.

References

Ilham, A. and Çiftçi, U. (2022) *Predicting the Success of Telemarketing Calls in the Banking Sector with Classification Models*. Available at: https://dergipark.org.tr/en/pub/aita/issue/70741/1137953

Ilham, A., Khikmah, L., Indra, I., Ulumuddin, U. and Iswara, I.B.A. (2019) *Long-term deposits prediction: A comparative framework of classification models for predicting the success of bank telemarketing*. Journal of Physics: Conference Series, 1175(1), 012035. Available at: https://iopscience.iop.org/article/10.1088/1742-6596/1175/1/012035/meta

Moro, S., Cortez, P. and Rita, P. (2014) *A data-driven approach to predict the success of bank telemarketing*. Lisbon: ISCTE – University Institute of Lisbon. Available at: https://core.ac.uk/display/55631291

Moro, S., Cortez, P. and Rita, P. (2022) *Bank Telemarketing: A Literature Review and A Data-Driven Model for Churn Prediction*. Available at: https://www.mdpi.com/1911-8074/15/6/269

Omar, N.A., Younis, M.N. and Rehman, A.A. (2023) *A data-driven predictive model for telemarketing subscription in the banking sector*. Available at: https://www.sciencedirect.com/science/article/abs/pii/S0360835222008622

Omar, S., Younis, M.B. and Rehman, F. (2023) *Predictive Analytics and Machine Learning in Direct Marketing for Anticipating Bank Term Deposit Subscriptions*. Available at: https://www.researchgate.net/publication/376585755

Quinlan, J. (1987) *Credit Approval* [Dataset]. UCI Machine Learning Repository. Available at: https://doi.org/10.24432/C5FS30

University of California, Irvine (UCI) Machine Learning Repository (2012) *Bank Marketing Data Set*. Available at: https://archive.ics.uci.edu/dataset/222/bank+marketing

Appendix

All supplementary materials supporting the transparency and reproducibility of this study are provided in the following appendices (A-B), including dataset descriptions, evaluation metric definitions, Python code for data preparation, detailed baseline results, and full threshold-optimisation tables.

Appendix A: Tables and Analytical Results

Appendix A provides the evaluation framework used throughout this report, including the confusion matrix structure, metric definitions, and all tabulated model results.

Appendix A1: Tables and Analytical Results

Variable Name	Role	Type	Demographic	Description
age	Feature	Integer	Age	Age of the client.
job	Feature	Categorical	Occupation	Type of job (admin., blue-collar, entrepreneur, housemaid, management, retired, self-employed, services, student, technician, unemployed, unknown).
marital	Feature	Categorical	Marital Status	Marital status (divorced, married, single, unknown; 'divorced' includes widowed).
education	Feature	Categorical	Education Level	(basic.4y, basic.6y, basic.9y, high.school, illiterate, professional.course, university.degree, unknown).
default	Feature	Binary		Has credit in default? (yes, no, unknown).
balance	Feature	Numeric		Average yearly

housing	Feature	Binary		Has a housing loan? (yes, no, unknown).
loan	Feature	Binary		Has a personal loan? (yes, no, unknown).
contact	Feature	Categorical		Contact communication type (cellular, telephone).
day_of_week	Feature	Categorical	Day of Week	Last contact day of the week (mon, tue, wed, thu, fri).
month	Feature	Categorical	Month	Last contact month of the year (jan, feb, mar, …, nov, dec).
duration	Feature	Integer		Last contact duration, in seconds. Important: should be excluded for realistic predictive modelling.
campaign	Feature	Integer		Number of contacts performed during this campaign and for this client.
pdays	Feature	Integer		Number of days that passed after the client was last contacted from a previous campaign (999 means never contacted).
previous	Feature	Integer		Number of contacts performed before this campaign.
poutcome	Feature	Categorical		Outcome of the previous marketing campaign (failure, nonexistent, success).

emp.var.rate	Feature	Numeric	Economic Indicator	Employment variation rate – quarterly indicator.
cons.price.idx	Feature	Numeric	Economic Indicator	Consumer price index – monthly indicator.
cons.conf.idx	Feature	Numeric	Economic Indicator	Consumer confidence index – monthly indicator.
euribor3m	Feature	Numeric	Economic Indicator	Euribor 3-month rate – daily indicator.
nr.employed	Feature	Numeric	Economic Indicator	Number of employees – quarterly indicator.
y	Target	Binary		Has the client subscribed a term deposit? (yes/no).

Appendix A2: Descriptive Statistics for All Variables

Variable	Count	Unique	Top	Freq	Mean	Std	Min	Max
age	41188	-	-		40.02	10.42	17	98
job	41188	12	admin.	10422	-	-	-	-
marital	41188	4	married	24928	-	-	-	-
education	41188	8	university.degree	12168	-	-	-	-
default	41188	3	no	32588	-	-	-	-
housing	41188	3	yes	21576	-	-	-	-
loan	41188	3	no	33950	-	-	-	-
contact	41188	2	cellular	26144	-	-	-	-
month	41188	10	may	13769	-	-	-	-
day_of_week	41188	5	thu	8623	-	-	-	-
campaign	41188	-	-	-	2.57	2.77	1	56
pdays	41188	-	-	-	962.47	186.91	0	999
previous	41188	-	-	-	0.17	0.49	0	7
poutcome	41188	3	nonexistent	35563	-	-	-	-
emp.var.rate	41188	-	-	-	0.08	1.57	-3.4	1.4
cons.price.idx	41188	-	-	-	93.58	0.58	92.20	94.77
cons.conf.idx	41188	-	-	-	-40.50	4.63	-50.80	-26.90
euribor3m	41188	-	-	-	3.62	1.73	0.63	5.05
nr.employed	41188	-	-	-	5167.04	72.25	4963.6	5228.1
y	41188	2	no	36548	-	-	-	-

Appendix B: Python Scripts

This section contains all essential Python scripts used to generate results, ensuring reproducibility and methodological transparency.

Import important libraries

```python
# Import the necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```python
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, recall_score, precision_score, classification_report, confusion_matrix
```

```python
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```python
# Import file
file_path = "/content/drive/MyDrive/Colab Notebooks/bank-additional.csv"
df = pd.read_csv(file_path,sep=';')
display(df.head())
```

Appendix B1: Exploratory Data Analysis (EDA)

Referenced in Section 2.1

Purpose: to produce the descriptive statistics table, histograms, and bar charts used to analyse variable distributions and class imbalance.

```python
# Statistics of outcome variable (y) and calculation of proportion
outcome_counts = df['y'].value_counts()
outcome_proportions = df['y'].value_counts(normalize=True)

print("Counts:")
display(outcome_counts)

print("\nProportions:")
display(outcome_proportions)
```

```
Counts:
```

y	count
no	36548
yes	4640

dtype: int64

```
Proportions:
```

y	proportion
no	0.887346
yes	0.112654

dtype: float64

```python
# Plot the distribution of the variable 'y' (target)
plt.figure(figsize=(6, 4))
sns.countplot(x='y', data=df, hue='y', palette='viridis', legend=False)
plt.title('Distribution of the variable y(Target)')
plt.xlabel('y (Outcome)')
plt.ylabel('Quantity')
plt.show()
```

```
# Select numeric columns to draw histograms and descriptive statistics tables.
numerical_cols = df.select_dtypes(include=np.number).columns
print("Descriptive statistics for numerical variables:")
display(df[numerical_cols].describe())
print("\nHistogram for digital deformations:")
df[numerical_cols].hist(figsize=(15, 10), bins=30)
plt.tight_layout()
plt.show()
```

```
# categorical data description (str format)
cat_cols = ['job', 'marital', 'education', 'default',
            'housing', 'loan', 'contact', 'month', 'day_of_week', 'poutcome']
n_cols = 3
n_rows = (len(cat_cols) + n_cols - 1) // n_cols

plt.figure(figsize=(n_cols * 5, n_rows * 4))

for i, col in enumerate(cat_cols):
    plt.subplot(n_rows, n_cols, i + 1)
    sns.countplot(x=col, data=df, palette='Set3',
                  order=df[col].value_counts().index)
    plt.title(f'Distribution of {col}')
    plt.xticks(rotation=45, ha='right')
    plt.xlabel('')
    plt.ylabel('Quantity')

plt.tight_layout()
plt.show()
```

Appendix B2: Data Cleaning and Feature Engineering

Referenced in Section 2.2

Includes all steps for data cleaning (replace unknowns, drop duration & default), feature transformations (log, binary, rare groups), one-hot encoding, and train-test split.

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```python
# 1) BASIC CLEANING
# - Replace 'unknown' -> NaN (to handle missing correctly)
# - Remove 'duration' column (to avoid data leakage)
# - Check for missing
df = df.replace('unknown', np.nan)
if 'duration' in df.columns:
    df = df.drop(columns=['duration'])
print("\nMissing after 'unknown'->NaN (top 10):")
print(df.isnull().sum().sort_values(ascending=False).head(10))
# 2) SPECIAL PROCESSING BY EDA
# - pdays: 999 -> NaN; create flag was_contacted_before (0/1)
# - Log-transform: campaign, previous -> reduce outlier
# - Type 'default' if almost constant
# - Group rare in job/education/poutcome

if 'pdays' in df.columns:
    df['pdays'] = df['pdays'].replace(999, np.nan)
    df['was_contacted_before'] = np.where(df['pdays'].isna(), 0, 1).astype(int)
for col in ['campaign', 'previous']:
    if col in df.columns:
        df[col + '_log'] = np.log1p(df[col])
# Type 'default' if > 95% of a value (almost no information)
if 'default' in df.columns:
    top_ratio = df['default'].value_counts(normalize=True, dropna=False).max()
    if top_ratio > 0.95:
        df = df.drop(columns=['default'])
# Rare clustering to reduce One-Hot (binary variable) sparsity
def lump_rare(series, min_freq=0.01):
    freq = series.value_counts(normalize=True)
    rare = freq[freq < min_freq].index
    return series.where(~series.isin(rare), other='other')
for c in ['job', 'education', 'poutcome']:
    if c in df.columns:
        df[c] = lump_rare(df[c], min_freq=0.01)
print("\nAfter feature engineering:")
print("Columns:", len(df.columns))
# 3) SPLIT TRAIN/TEST (stratify=y)
X = df.drop(columns=['y'])
y = (df['y'] == 'yes').astype(int)
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y)
print("\nTrain/Test shapes:", X_train.shape, X_test.shape)
print("Target distribution (train):")
print(y_train.value_counts(normalize=True))
num_cols = X_train.select_dtypes(exclude=['object']).columns.tolist()
cat_cols = X_train.select_dtypes(include=['object']).columns.tolist()
print("\nNumeric cols ({}):".format(len(num_cols)), num_cols)
print("Categorical cols ({}):".format(len(cat_cols)), cat_cols)

num_imputer = SimpleImputer(strategy='median')
cat_imputer = SimpleImputer(strategy='most_frequent')

X_train_num = pd.DataFrame(num_imputer.fit_transform(X_train[num_cols]),
    columns=num_cols, index=X_train.index)
X_test_num = pd.DataFrame(num_imputer.transform(X_test[num_cols]),
    columns=num_cols, index=X_test.index)

X_train_cat = pd.DataFrame(cat_imputer.fit_transform(X_train[cat_cols]),
    columns=cat_cols, index=X_train.index)
X_test_cat = pd.DataFrame(cat_imputer.transform(X_test[cat_cols]),
    columns=cat_cols, index=X_test.index)
# 4) SCALE NUMERIC
scaler = StandardScaler()
X_train_num_scaled = pd.DataFrame(scaler.fit_transform(X_train_num),
    columns=num_cols, index=X_train.index)
X_test_num_scaled = pd.DataFrame(scaler.transform(X_test_num),
```

```python
X_test_num_scaled = pd.DataFrame(scaler.transform(X_test_num),
    columns=num_cols, index=X_test.index)
# 5) ONE-HOT ENCODE CATEGORICAL
try:
    ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
except TypeError:
    ohe = OneHotEncoder(handle_unknown='ignore', sparse=False)
X_train_cat_encoded = pd.DataFrame(ohe.fit_transform(X_train_cat),
    columns=ohe.get_feature_names_out(cat_cols),
    index=X_train.index)
X_test_cat_encoded = pd.DataFrame(ohe.transform(X_test_cat),
    columns=ohe.get_feature_names_out(cat_cols),
    index=X_test.index)
# Merge processed data
X_train_prepared = pd.concat([X_train_num_scaled, X_train_cat_encoded], axis=1)
X_test_prepared  = pd.concat([X_test_num_scaled,  X_test_cat_encoded],  axis=1)
print("\n PREPROCESSING DONE")
print("X_train_prepared:", X_train_prepared.shape)
print("X_test_prepared :", X_test_prepared.shape)
```

```
Missing after 'unknown'->NaN (top 10):
default        8597
education      1731
housing         990
loan            990
job             330
marital          80
age               0
contact           0
month             0
day_of_week       0
dtype: int64

After feature engineering:
Columns: 23

Train/Test shapes: (32950, 22) (8238, 22)
Target distribution (train):
y
0    0.887344
1    0.112656
Name: proportion, dtype: float64

Numeric cols (12): ['age', 'campaign', 'pdays', 'previous', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx', 'euribor3m', 'nr.employed', 'was_contacte
Categorical cols (10): ['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'day_of_week', 'poutcome']

✅ PREPROCESSING DONE
X_train_prepared: (32950, 59)
X_test_prepared : (8238, 59)
```

Appendix B3: Model Design, Optimisation, and Cost-Sensitive Threshold Tuning

Referenced in Sections 2.3, 3.2, and 3.3

B3.1: code for Model Design

```
# Decision Tree (baseline)
dt_base = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=4,
    min_samples_leaf=7,
    random_state=42)
dt_base.fit(X_train_prepared, y_train)
# Logistic Regression (baseline)
log_base = LogisticRegression(
    max_iter=2000,
    solver='lbfgs',
    random_state=42)
log_base.fit(X_train_prepared, y_train)
print("Trained baseline: DT_base & LR_base")
```

```
Trained baseline: DT_base & LR_base
```

B3.2: Code for ROC CURVE (baseline)

```
# ROC CURVE (BASELINE)
import matplotlib.pyplot as plt
from sklearn.metrics import RocCurveDisplay
plt.figure(figsize=(7,6))
RocCurveDisplay.from_predictions(y_test, y_proba_dt_base,  name="DT (baseline, entropy)")
RocCurveDisplay.from_predictions(y_test, y_proba_log_base, name="LR (baseline)")
plt.plot([0,1], [0,1], 'k--', lw=1)
plt.title("ROC Curve - Baseline Model")
plt.xlabel("False Positive Rate (FPR)")
plt.ylabel("True Positive Rate (TPR - Recall)")
plt.legend(loc="lower right")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

B3.3: Code for Model Evaluation Function

Purpose:

This code defines a reusable evaluation function named eval_model, which was used

throughout the project to measure and compare model performance consistently. The function calculates key classification metrics such as Accuracy, Precision, Recall, F1-score, ROC-AUC, and PR-AUC, and generates a confusion matrix to summarise prediction outcomes. It also supports cost-sensitive evaluation by incorporating campaign-related weights for false positives and false negatives, allowing direct assessment of marketing efficiency.

The eval_model function was applied in Sections 3.1, 3.2, and 3.3 to evaluate both baseline and optimised models under identical conditions.

```python
COST_FP = 1   # 1 wasted call
COST_FN = 4   # 1 missed lead
def eval_model(y_true, y_pred, y_proba, name):
    # core metrics
    acc  = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec  = recall_score(y_true, y_pred, zero_division=0)
    f1   = f1_score(y_true, y_pred, zero_division=0)

    try:
        roc_auc = roc_auc_score(y_true, y_proba)
    except ValueError:
        roc_auc = np.nan
    try:
        pr_auc  = average_precision_score(y_true, y_proba)
    except ValueError:
        pr_auc = np.nan

    # confusion matrix & rates
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()
    tpr = tp / (tp + fn) if (tp + fn) > 0 else 0.0
    tnr = tn / (tn + fp) if (tn + fp) > 0 else 0.0
    fpr = fp / (fp + tn) if (fp + tn) > 0 else 0.0
    fnr = fn / (fn + tp) if (fn + tp) > 0 else 0.0
    bal_acc = (tpr + tnr) / 2 if (tpr + tnr) > 0 else 0.0
    # business cost
    total_cost = COST_FP * fp + COST_FN * fn
    # display
```

```python
    print(f"\n=== {name} ===")
    print(f"Accuracy       : {acc:.4f}")
    print(f"Precision      : {prec:.4f}")
    print(f"Recall (TPR)   : {rec:.4f}")
    print(f"F1-score       : {f1:.4f}")
    print(f"ROC AUC        : {roc_auc:.4f}")
    print(f"PR  AUC        : {pr_auc:.4f}")
    print(f"Specificity(TNR): {tnr:.4f} | FPR: {fpr:.4f} | FNR: {fnr:.4f}")
    print(f"Balanced Acc   : {bal_acc:.4f}")
    print("Confusion Matrix:\n", cm)
    print(f"TP={tp}, FP={fp}, FN={fn}, TN={tn}")
    print(f"Support (pos={tp+fn}, neg={tn+fp})")
    print(f"Estimated Cost : FP*{COST_FP} + FN*{COST_FN} = {total_cost}")
    return {
        'Model': name,
        'Accuracy': acc,
        'Precision': prec,
        'Recall': rec,
        'F1': f1,
        'ROC_AUC': roc_auc,
        'PR_AUC': pr_auc,
        'Specificity': tnr,
        'FPR': fpr,
        'FNR': fnr,
        'Balanced_Acc': bal_acc,
        'TP': tp, 'FP': fp, 'FN': fn, 'TN': tn,
        'Support_Pos': tp + fn,
        'Support_Neg': tn + fp,
        'Cost': total_cost
    }
# Predict and evaluate 2 baseline models
y_pred_dt_base  = dt_base.predict(X_test_prepared)
y_proba_dt_base = dt_base.predict_proba(X_test_prepared)[:, 1]
y_pred_log_base  = log_base.predict(X_test_prepared)
y_proba_log_base = log_base.predict_proba(X_test_prepared)[:, 1]
results_base = []
results_base.append(eval_model(y_test, y_pred_dt_base,  y_proba_dt_base,  "Decision Tree (baseline, entropy)"))
results_base.append(eval_model(y_test, y_pred_log_base, y_proba_log_base, "Logistic Regression (baseline)"))
results_base_df = pd.DataFrame(results_base)
display(results_base_df.sort_values(['Balanced_Acc','PR_AUC','ROC_AUC'], ascending=False))
```

B3.4: code for Grid Search Optimisation

```python
# GRID SEARCH (DECISION TREE, ENTROPY)
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(criterion='entropy', random_state=42)
param_grid_dt = {
    'max_depth': [4, 6, 8, 12, 16],
    'min_samples_leaf': [2, 3, 5, 8, 15, 30],
    'min_samples_split': [5, 10, 20, 40, 80],
    'class_weight': ['balanced'],
    'ccp_alpha': [0.0, 0.0002, 0.0008, 0.002, 0.003]
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

grid_dt = GridSearchCV(
    estimator=dt,
    param_grid=param_grid_dt,
    scoring='roc_auc',
    cv=cv,
    n_jobs=-1,
    verbose=1
)
grid_dt.fit(X_train_prepared, y_train)
best_tree = grid_dt.best_estimator_
print("\n GridSearch DT done")
print("Best params:", grid_dt.best_params_)
print("Best CV ROC-AUC:", grid_dt.best_score_)
# Test set
from sklearn.metrics import roc_auc_score
y_proba_dt_best = best_tree.predict_proba(X_test_prepared)[:, 1]
y_pred_dt_best  = (y_proba_dt_best >= 0.5).astype(int)
print("\n=== Decision Tree (best on test, th=0.5) ===")
_ = eval_model(y_test, y_pred_dt_best, y_proba_dt_best, "DT (best, entropy)")
```

```python
# GRID SEARCH (LOGISTIC REGRESSION)
from sklearn.linear_model import LogisticRegression
param_grids_log = [
    {   'solver': ['liblinear'],
        'penalty': ['l1', 'l2'],
        'C': [0.05, 0.1, 0.5, 1.0, 2.0, 5.0],
        'class_weight': [None, 'balanced'],
        'max_iter': [2000],
    },
    {   'solver': ['lbfgs'],
        'penalty': ['l2'],
        'C': [0.05, 0.1, 0.5, 1.0, 2.0, 5.0],
        'class_weight': [None, 'balanced'],
        'max_iter': [2000],
    },
    {   'solver': ['saga'],
        'penalty': ['l1', 'l2'],
        'C': [0.05, 0.1, 0.5, 1.0, 2.0, 5.0],
        'class_weight': [None, 'balanced'],
        'max_iter': [2000],
},
]
grid_log = GridSearchCV(
    estimator=LogisticRegression(),
    param_grid=param_grids_log,
    scoring='roc_auc',
    cv=cv,
    n_jobs=-1,
    verbose=1)
grid_log.fit(X_train_prepared, y_train)
best_log = grid_log.best_estimator_
print("\n GridSearch Logistic done")
print("Best params:", grid_log.best_params_)
print("Best CV ROC-AUC:", grid_log.best_score_)
y_proba_log_best = best_log.predict_proba(X_test_prepared)[:, 1]
y_pred_log_best  = (y_proba_log_best >= 0.5).astype(int)
print("\n=== Logistic Regression (best on test, th=0.5) ===")
_ = eval_model(y_test, y_pred_log_best, y_proba_log_best, "Logistic (best)")
```

B3.5: code for Cost-Sensitive Threshold Tuning

```
# COST-SENSITIVE THRESHOLD TUNING
# Cost configuration
COST_FP = 2.0
COST_FN = 15.0
COST_TP = -4.0
COST_TN = 0.0
def expected_cost(y_true, y_proba, threshold=0.5):
    y_pred = (y_proba >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    total_cost = tp*COST_TP + fp*COST_FP + fn*COST_FN + tn*COST_TN
    return total_cost / len(y_true)
def sweep_thresholds(y_true, y_proba, name):
    thresholds = np.linspace(0.01, 0.99, 99)
    rows = []
    for t in thresholds:
        rows.append({'threshold': t, 'avg_cost': expected_cost(y_true, y_proba, t)})
    df_cost = pd.DataFrame(rows).sort_values('avg_cost')
    print(f"\nTop 5 threshold (min avg cost) — {name}")
    display(df_cost.head(5))
    best_t  = float(df_cost.iloc[0]['threshold'])
    best_ac = float(df_cost.iloc[0]['avg_cost'])
    return best_t, best_ac, df_cost
dt_best_th,  dt_best_cost,  dt_cost_df  = sweep_thresholds(y_test, y_proba_dt_best,  "DT (best)")
log_best_th, log_best_cost, log_cost_df = sweep_thresholds(y_test, y_proba_log_best, "Logistic (best)")
```

B3.6: Code for picking a winner model

```
# PICK WINNER (Base on AVG COST)
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

def metrics_at(y_true, y_proba, th, name):
    y_pred = (y_proba >= th).astype(int)
    return {
        'Model': name,
        'Threshold': th,
        'Accuracy': accuracy_score(y_true, y_pred),
        'Precision': precision_score(y_true, y_pred, zero_division=0),
        'Recall': recall_score(y_true, y_pred, zero_division=0),
        'F1': f1_score(y_true, y_pred, zero_division=0),
        'ROC_AUC': roc_auc_score(y_true, y_proba),
        'Avg_Cost': expected_cost(y_true, y_proba, th)
    }

summary = pd.DataFrame([
    metrics_at(y_test, y_proba_dt_best,  dt_best_th,  "DT (best @th*)"),
    metrics_at(y_test, y_proba_log_best, log_best_th, "Logistic (best @th*)")
]).sort_values('Avg_Cost')
display(summary)

winner_row = summary.iloc[0]
print(f"\n WINNER: {winner_row['Model']}  @ threshold={winner_row['Threshold']:.3f}  "
      f"(Avg Cost={winner_row['Avg_Cost']:.6f})")
```

	Model	Threshold	Accuracy	Precision	Recall	F1	ROC_AUC	Avg_Cost
0	DT (best @th*)	0.41	0.821801	0.351322	0.687500	0.465015	0.807844	0.504249
1	Logistic (best @th*)	0.45	0.802015	0.322025	0.685345	0.438167	0.801104	0.547949

```
 WINNER: DT (best @th*)  @ threshold=0.410  (Avg Cost=0.504249)
```

B3.7: Code for Roc curve comparison

```
# ROC CURVE (BEST MODELS)
import matplotlib.pyplot as plt
from sklearn.metrics import RocCurveDisplay
# ROC Plot for Decision Tree (Baseline vs Best)
plt.figure(figsize=(7,6))
RocCurveDisplay.from_predictions(y_test, y_proba_dt_base, name="DT (baseline)",  linestyle=':')
RocCurveDisplay.from_predictions(y_test, y_proba_dt_best,  name="DT (best, entropy)")
plt.plot([0,1], [0,1], 'k--', lw=1)
plt.title("ROC Curve — Decision Tree: Best vs Baseline")
plt.xlabel("False Positive Rate (1 - Specificity)")
plt.ylabel("True Positive Rate (Recall)")
plt.legend(loc="lower right")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
# ROC Plot for Logistic Regression (Baseline vs Best)
plt.figure(figsize=(7,6))
RocCurveDisplay.from_predictions(y_test, y_proba_log_base, name="Logistic (baseline)", linestyle=':')
RocCurveDisplay.from_predictions(y_test, y_proba_log_best, name="Logistic (best)")
plt.plot([0,1], [0,1], 'k--', lw=1)
plt.title("ROC Curve — Logistic Regression: Best vs Baseline")
plt.xlabel("False Positive Rate (1 - Specificity)")
plt.ylabel("True Positive Rate (Recall)")
plt.legend(loc="lower right")
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

B3.6: Code for Interpretation and Feature Insight

```
# Show only meaningful features (importance > threshold)
threshold = 0.01
fi_sig = fi[fi > threshold].sort_values(ascending=False)

if fi_sig.empty:
    print(f"No features exceed the threshold ({threshold}) — showing top 20 instead.")
    fi_sig = fi.head(20)

plt.figure(figsize=(8, 6))
fi_sig.sort_values().plot(kind='barh', color="#3A7CA5")
plt.title(f"Feature Importances > {threshold:.3f} — Decision Tree (best)")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()

print(f"\n Showing {len(fi_sig)} significant features (importance > {threshold}).")
display(fi_sig)
```

```
# Logistic Regression: Coefficient Bars
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
try:
    model_log = best_log
except NameError:
    model_log = log_base

coefs = pd.Series(model_log.coef_[0], index=X_train_prepared.columns)
top_pos = coefs.sort_values(ascending=False).head(15)
top_neg = coefs.sort_values(ascending=True).head(15)
fig, axes = plt.subplots(1, 2, figsize=(14, 6), sharex=False)
top_pos.sort_values().plot(kind='barh', ax=axes[0])
axes[0].set_title("Top positive coefficient (increases probability of 'yes')")
axes[0].set_xlabel("Coefficient (+)")
top_neg.sort_values().plot(kind='barh', ax=axes[1], color='red')
axes[1].set_title("Top negative coefficient (decreases probability of 'yes')")
axes[1].set_xlabel("Coefficient (-)")
plt.tight_layout()
plt.show()
```